Michał Radecki Lab 2 SAK

```html
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <title>Zegar Analogowy - Lab 02</title>
    <style>
        body {
            display: flex;
            flex-direction: column;
            align-items: center;
            justify-content: center;
            height: 100vh; margin: 0;
            background: #919a5b;
            color: white;
            font-family: sans-serif;
        }
        canvas { background: #fff;
             border-radius: 50%;
              border: 15px solid #8f8756;
               box-shadow: 0 10px 30px rgba(0,0,0,0.5); }
        .info { margin-top: 20px; color: #bdc3c7; }
    </style>
</head>
<body>
    <canvas id="clockCanvas" width="500" height="500"></canvas>
    <div class="info">Naciśnij <strong>SPACJĘ</strong>, aby zatrzymać czas</div>

    <script type="module" src="main.js"></script>
</body>
</html>
```

JS main.js > ...

```js
import { Clock } from './Clock.js';

const clock = new Clock('clockCanvas');

function animate() {
    clock.update();
    requestAnimationFrame(animate); // Wywołanie kolejnej klatki
}

window.addEventListener('keydown', (e) => {
    if (e.code === 'Space') {
        clock.togglePause();
    }});

animate();

```

```js
import { Hand } from './Hand.js';

export class Clock {
    constructor(canvasId) {
        this.canvas = document.getElementById(canvasId);
        this.ctx = this.canvas.getContext('2d');
        this.isPaused = false;

        this.hourHand = new Hand(100, 8, '#2c3e50');
        this.minuteHand = new Hand(150, 5, '#34495e');
        this.secondHand = new Hand(180, 2, '#e74c3c');
    }

    drawFace() {
        const ctx = this.ctx;
        ctx.save(); // Zapisujemy stan przed rysowaniem tarczy
        ctx.strokeStyle = '#2c3e50';

        for (let i = 0; i < 60; i++) {
            const isHour = i % 5 === 0;
            ctx.lineWidth = isHour ? 4 : 1;

            ctx.beginPath();
            ctx.moveTo(0, -210);
            ctx.lineTo(0, isHour ? -230 : -220);
            ctx.stroke();

            ctx.rotate(Math.PI / 30); // Obrót o 6 stopni (360/60)
        }
        ctx.restore(); // Przywracamy stan po narysowaniu tarczy
    }
```

```
    update() {
        if (this.isPaused) return;

        const now = new Date();
        const ms = now.getMilliseconds();
        const sec = now.getSeconds() + ms / 1000;
        const min = now.getMinutes() + sec / 60;
        const hr = (now.getHours() % 12) + min / 60;

        // Czyszczenie tła
        this.ctx.clearRect(0, 0, this.canvas.width, this.canvas.height);

        this.ctx.save();
        this.ctx.translate(this.canvas.width / 2, this.canvas.height / 2);

        this.drawFace();

        // Rysowanie wskazówek
        this.hourHand.draw(this.ctx, (hr / 12) * 2 * Math.PI);
        this.minuteHand.draw(this.ctx, (min / 60) * 2 * Math.PI);
        this.secondHand.draw(this.ctx, (sec / 60) * 2 * Math.PI);

        this.ctx.restore();
    }

    togglePause() {
        this.isPaused = !this.isPaused;
    }
}
```

```javascript
export class Hand {
    constructor(length, width, color) {
        this.length = length;
        this.width = width;
        this.color = color;
    }

    draw(ctx, angle) {
        ctx.save(); // Zapisanie stanu przed rotacją

        ctx.rotate(angle);
        ctx.beginPath();
        ctx.strokeStyle = this.color;
        ctx.lineWidth = this.width;
        ctx.lineCap = "round";

        ctx.moveTo(0, 0);
        ctx.lineTo(0, -this.length);
        ctx.stroke();

        // Kółeczko dekoracyjne
        ctx.beginPath();
        ctx.arc(0, 0, 5, 0, Math.PI * 2);
        ctx.fillStyle = this.color;
        ctx.fill();

        ctx.restore();
    }
}
```

Naciśnij **SPACJĘ**, aby zatrzymać czas